# ACID For Data Engineers

How well do you know about Atomicity, Consistency, Isolation and Durability?

VU TRINH

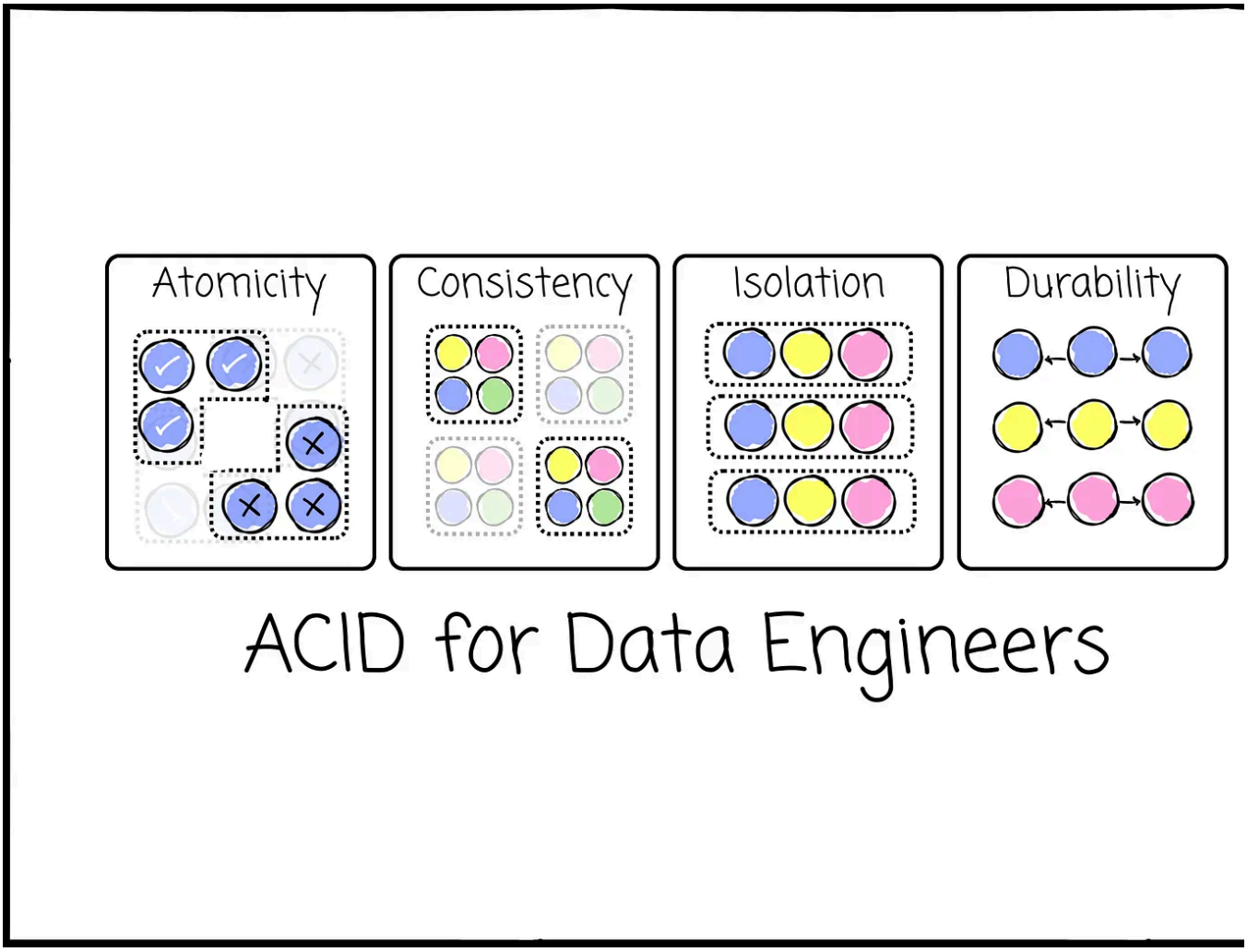
JUL 15, 2025 • PAID

7

Sl

> *I will publish a paid article every Tuesday. I wrote these with one goal in mind: to offe readers, whether they are feeling overwhelmed when beginning the journey or seeking a deeper understanding of the field, 15 minutes of practical lessons and insights on nearl everything related to data engineering.*
>
> *I invite you to join the club with a* ***50% discount on the yearly package.*** *Let's not be suc data engineering together.*

Type your email... Subscribe




# Intro

Like many other guys who are interested in data management systems, I heard and learned about ACID guarantees. However, I find that internet resources on this to are quite oversimplified, and I decided to invest my time in researching it.

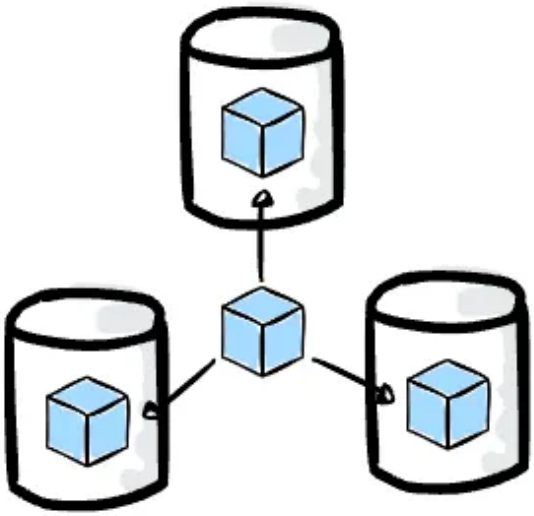
This article is my note on the relational database ACID constraints. I hope that th next time we read a document about an OLAP database, when it mentions that it supports ACID, we will understand what it is referring to.

> Note: This article is written for my newsletter audience, who are primarily engineers, so that the content will focus more on the ACID properties of OLAP systems.

# Durability

For me, this property is the easiest one to understand. It ensures that once the transaction commits successfully, all of its changes will never be lost in any circumstances.

Most OLAP systems are multi-node, as they operate across multiple servers. Durability is ensured by successfully storing various data copies on more than one server.



When building the data management system on object storage (e.g., S3 or GCS), th services will guarantee this property without requiring us to implement it explicit Amazon S3 and Google Cloud storage both provide 99.999999999% durability.

If you're using Hudi, Iceberg, Delta Lake, or commercial solutions like Redshift, Snowflake, or Databricks—all built on object storage—you usually don't need to w about durability.

# Isolation

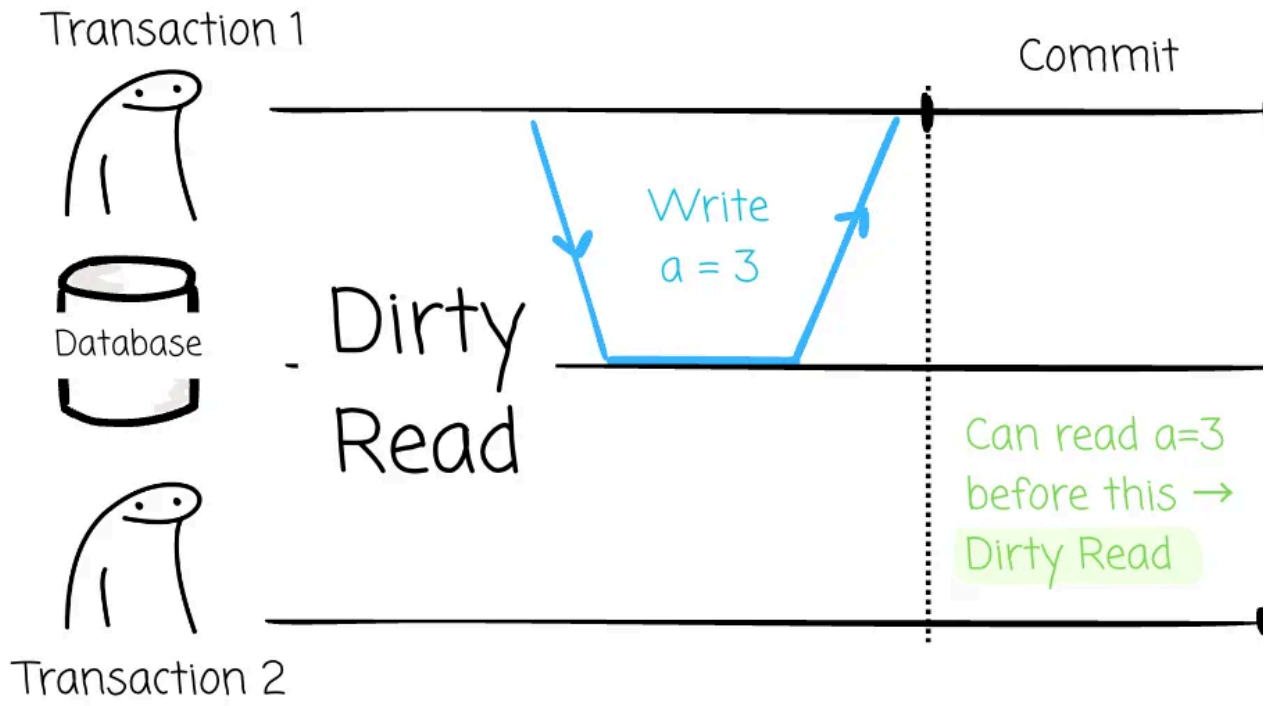
People use databases to store data. Then, the data is accessed by those who need it They want, at the very least, the data to be accurate (it reflects what happens in rea life), especially in an environment where potentially many clients use the database

Isolation ensures that concurrent transactions are isolated. In other words, a transaction can think that it is the only one running in the database. It is straightforward if the two running transactions are working on different pieces of data; let them run, and everything will be fine. However, things are more difficult when operating on the same data.

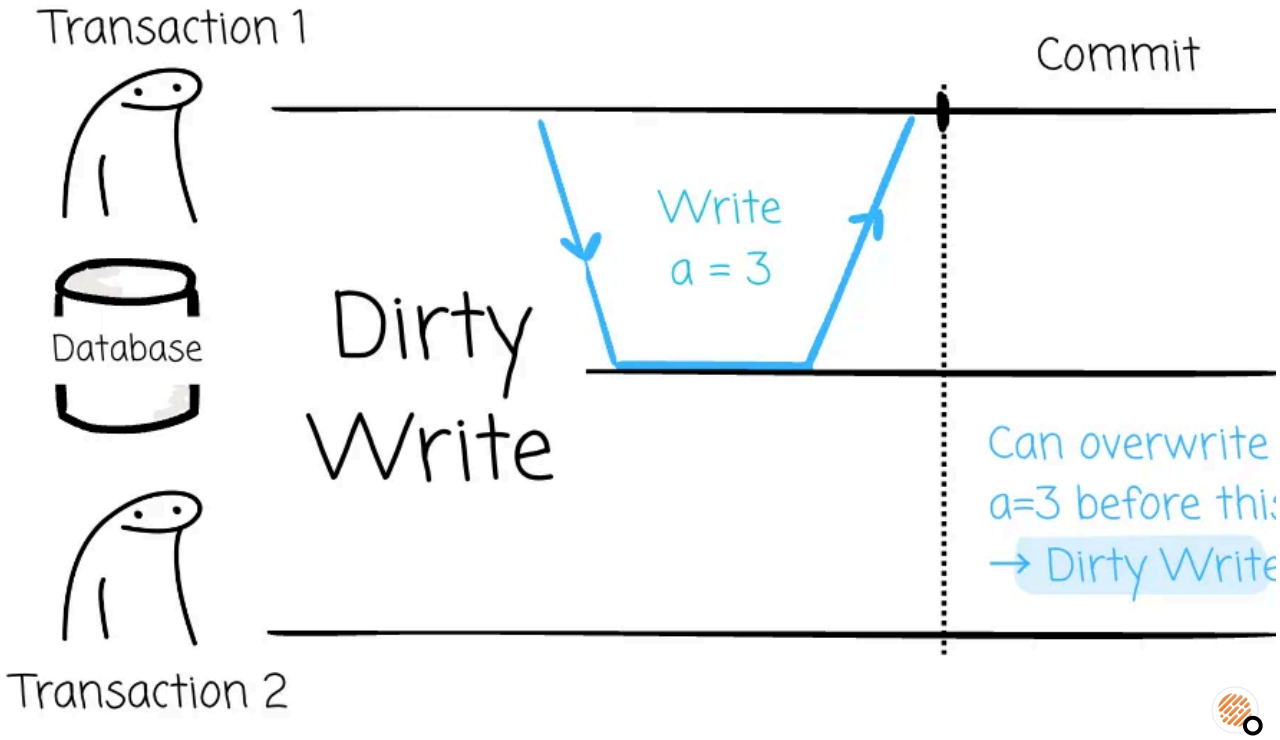
## Anomalies

Over time, database researchers have identified a set of concurrency anomalies (potential concurrent problems). Imagine we have two concurrent transactions, T1 T2, that operate on the same piece of data. These anomalies are:

- **Dirty Read:** T1 writes some data to the database, but it hasn't committed the changes yet. If T2 can read these uncommitted changes, this behavior is called dirty read.




- **Dirty Write:** T1 writes some data to the database, but it hasn't committed the changes yet. If T2 can overwrite these uncommitted changes, this behavior is referred to as a dirty write.




- **Non-repeatable read:** During the transaction, if T1 sees a piece of data with a different value at a different point in time, this behavior is called the non-repeatable read.

- **Lost Update:** T1 reads a piece of data (a = 2), and T2 also reads the same data. then modifies the object and commits (a = 3). T2 then modifies this data based its original read (a = 2) and commits (with a new value of a = 4), accidentally overwriting T1's update.
- **Phantom read:** T1 executes a read operation with objects that satisfy a particu WHERE clause. T2 then modifies the objects that satisfy this WHERE clause commits changes. If T1 repeats its query, it sees a different result.
    - This is different from **Non-repeatable read** and **Dirty Read** because the T and T2 operate on different objects.
- **Write Skew:** It can happen when T1 reads some data from the A object, make changes based on the value it saw, and commits the changes to the B object. However, during the process of making the B's changes, the original values of are changed by the T2.
    - This is different from **Lost Update** and **Dirty Write** because the T1 and T operate on different objects.
    - The effect of changing the filter conditions during the process of making changes based on the conditions is also referred to as **phantoms**.

> ***Note**: I will provide concrete examples in the following sections for those that do not se obvious at first glance: the **non-repeatable read**, the **lost update**, the **phantom read**, and **write skew**.*

To deal with these anomalies, database researchers have introduced the following isolation levels, from looser to tighter:

> *For example, read committed can prevent dirty reads and dirty writes; thus, snapshot isolation can also prevent these anomalies.*

- Read committed
- Snapshot isolation
- Serializability

## Read committed

This level ensures that you're always reading and writing data that is committed (r dirty reads and dirty writes).